ASSIGNMENT 10.1

D.Charmi

2303A51314

Batch – 05

TASK – 01:

Prompt : #Correct the following code so that it prints average score of a student

'''def calc_average(marks):

total = 0

for m in marks:

total += m

average = total / len(marks)

return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks)'''

Code :

```
#Correct the following code so that it prints average score of a student
'''def calc_average(marks):
total = 0
for m in marks:
total += m
average = total / len(marks)
return avrage # Typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)'''

# Corrected code
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return average # Corrected typo here
```

```
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks))
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop
ing/10.1.py"
Average Score is  86.25
PS C:\Users\devis\OneDrive\Desktop
```

Explanation :

Indentation: The original code had incorrect indentation, which would lead to a syntax error. The corrected code properly indents the lines within the function.

Typo Correction: The original code had a typo in the return statement ("avrage" instead of "average"). This has been corrected to ensure the function returns the correct variable.

Added Missing Parenthesis: The original code was missing a closing parenthesis in the print statement. This has been added to ensure the code runs without syntax errors.

TASK – 02:

Prompt :

Format the below code properly which follows pep8 guidelines, add docstrings and add comments to explain the code.

'''def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))'''

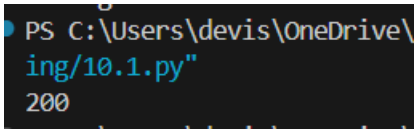
Code :

```
# Format the below code properly which follows pep8 guidelines, add docstrings
and add comments to explain the code.
'''def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))'''
#Modified code with proper formatting, docstrings, and comments
def area_of_rect(length, breadth):
    """
    Calculate the area of a rectangle.
```

```
    Parameters:
    length (float): The length of the rectangle.
    breadth (float): The breadth of the rectangle.

    Returns:
    float: The area of the rectangle.
    """
    return length * breadth
# Example usage
print(area_of_rect(10, 20))  # Output: 200
```

Output :



Explanation :

The function `area_of_rect` takes two parameters, `length` and `breadth`, which represent the dimensions of the rectangle. It calculates the area by multiplying these two values and returns the result. The example usage demonstrates how to call the function with specific values for length and breadth, and it prints the calculated area.

TASK – 03:

Prompt :

#Add proper variable names, inline comments and enhance readability of the the below code

'''def c(x,y):

return x*y/100

a=200

b=15

print(c(a,b))'''

Code :

```
#Add proper variable names, inline comments and enhance readability of the the
below code
'''def c(x,y):
return x*y/100
a=200
b=15
print(c(a,b))'''

#Enhanced version of the code with proper variable names, inline comments, and
improved readability
def calculate_percentage(value, percentage):
    """
    Calculate the percentage of a given value.

    Parameters:
    value (float): The total value from which the percentage is to be
calculated.
    percentage (float): The percentage to be calculated.

    Returns:
    float: The calculated percentage of the value.
    """
    return value * percentage / 100
# Define the total value and the percentage to be calculated
total_value = 200
percentage_to_calculate = 15
# Calculate and print the result
result = calculate_percentage(total_value, percentage_to_calculate)
print(result)  # Output: 30.0
```

Output :

```
PS C:\Users\devis\One
ing/10.1.py"
30.0
```

Explanation :

The function `calculate_percentage` takes two parameters: `value`, which is the total value, and `percentage`, which is the percentage to be calculated. It returns the result of the calculation by multiplying the total value with the percentage and dividing by 100. The variable names are descriptive, and inline comments explain the purpose of each part of the code, enhancing readability.

TASK – 04:

Prompt :

#Optimize the below code by reusing the function and removing the redundant code

'''students = ["Alice", "Bob", "Charlie"]

print("Welcome", students[0])

print("Welcome", students[1])

print("Welcome", students[2])'''

Code :

```
#Optimize the below code by reusing the function and removing the redundant
code
'''students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])'''

students = ["Alice", "Bob", "Charlie"]
def welcome_students(students):
    for student in students:
        print("Welcome", student)
welcome_students(students)
```

Output :

```
PS C:\Users\devis\OneDrive\Desktop\AI As
ing/10.1.py"
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\devis\OneDrive\Desktop\AI As
```

Explanation :

The original code had redundant print statements for each student. By creating a function `welcome_students`, we can reuse the code to welcome each student in the list, making it more efficient and easier to maintain. The function iterates through the list of students and prints a welcome message for each one, eliminating the need for multiple print statements.

TASK – 05:

Prompt :

#Use list comprehension or vectorized operations to improve performance and decrease time complexity for the below code

'''nums = [i for i in range(1,1000000)]

squares = []

for n in nums:

squares.append(n**2)

print(len(squares))'''

Code :

```
#Use list comprehension or vectorized operations to improve performance and
decrease time complexity for the below code
'''nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
squares.append(n**2)
print(len(squares))'''
#Optimized version using list comprehension:
nums = [i for i in range(1,1000000)]
squares = [n**2 for n in nums]
print(len(squares))
```

Output :

```
PS C:\Users\devis\OneDrive
ing/10.1.py"
999999
PS C:\Users\devis\OneDrive
```

Explanation :

The original code uses a for loop to iterate through each number in the list and appends its square to the 'squares' list, resulting in O(n) time complexity. The optimized version uses a list comprehension, which is more concise and can be faster than a traditional for loop due to internal optimizations in Python. Both versions have the same time complexity of O(n), but the list comprehension is generally more efficient and easier to read.

TASK – 06:

Prompt : #Refactor the below code by using elif instead of multiple else blocks to improve readability and maintainability.

'''if score >= 90:

return "A"

else:

if score >= 80:

return "B"

else:

if score >= 70:

return "C"

else:

if score >= 60:

return "D"

else:

return "F"'''

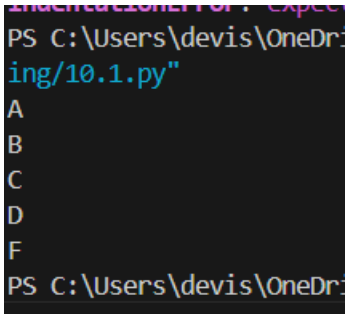
Code :

```
#Refactor the below code by using elif instead of multiple else blocks to
improve readability and maintainability.

'''if score >= 90:
return "A"
else:
if score >= 80:
return "B"
else:
if score >= 70:
return "C"
else:
if score >= 60:
return "D"
else:
return "F"'''
def grade(score):
    if score >= 90:
```

```
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
# Example usage
print(grade(95))  # Output: A
print(grade(85))  # Output: B
print(grade(75))  # Output: C
print(grade(65))  # Output: D
print(grade(55))  # Output: F
```

Output :



Explanation :

Using elif improves readability by clearly indicating that the conditions are mutually exclusive and part of the same decision structure. It reduces the nesting of code, making it easier to follow the logic. Additionally, it enhances maintainability by allowing for easier modifications in the future; if you need to add or change a condition, you can do so without affecting the overall structure of the code.